



BENNETT
UNIVERSITY
A TIMES GROUP INITIATIVE

BENNETT UNIVERSITY

School of Computer Science Engineering and Technology

Computational Thinking and Programming

Course Code: 2026CSAE100

PYTHON PROJECT REPORT

Schedailor: Dynamic Student Schedule Generator

Student Name	Agraj Gaur
Enrollment Number	S26CSEU0796
Batch / Section	B - 23 / S - 12
Team Member 2	Shreyan Gambhir
Team Member 3	Priyanshu Kumar Bhardwaj
Team Member 4	Bhupesh Singh
Team Member 5	Abhay Singh

Academic Year: 2026–27

1 Problem Statement

- What is the problem?

University students and corporate professionals struggle to create an optimized, tailor-made daily schedule that provides maximum efficiency and comfortability while managing unpredictable organizational changes.

- Who faces this problem?

Primarily college students (both day scholars and hostellers) and corporate sector employees

- Why is it useful to solve this problem?

By dynamically tracking subjective inputs (like sleep consistency, varying meal times, and transit), the system reduces burnout, ensures tasks are completed according to priority/deadlines, and adapts to the user's toughness preferences.

2 Project Objectives

Write 3–5 clear objectives of your project.

1. Capture detailed subjective user inputs (wake times, transit status, eating/break frequencies, and unpredictable extra events).
2. Store and organize user tasks based on Duration, Toughness level (1-10), and Priority level (1-10).
3. Assign tasks to specific times of the day based on user study preferences (e.g., moving from easy to difficult tasks or vice versa).
4. Generate and output a newly optimized, efficient schedule tailored specifically to the user's daily availability.

3 Main Features / Modules

List the main features or modules developed in the project.

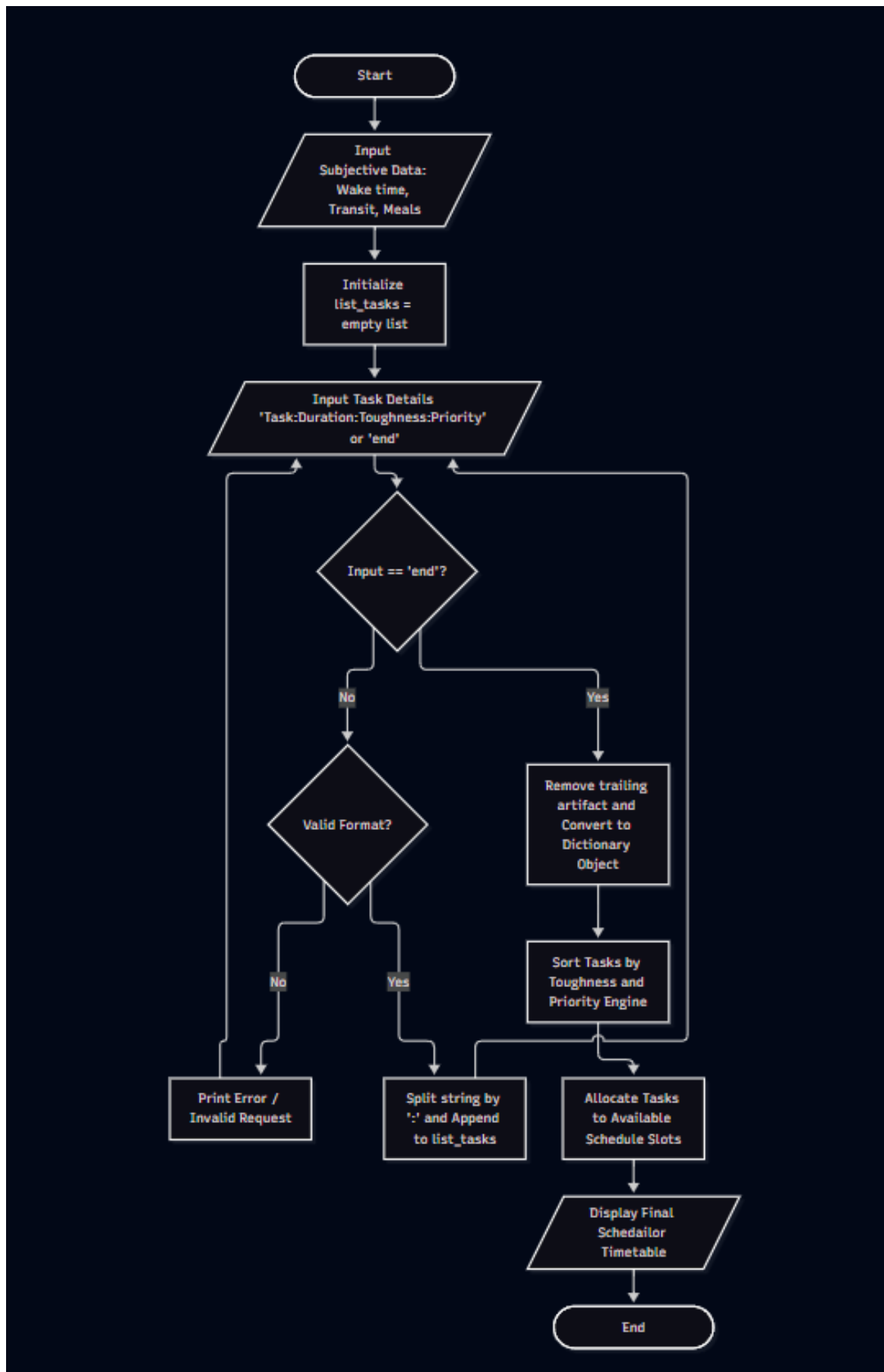
S. No.	Module / Feature	Description
1	Subjective Input Processor	Captures student lifestyle data (day scholar vs. hosteller, sleep/wake times, meal durations, and break frequency).
2	Task Collection Loop	A while loop interface allowing users to continuously input tasks formatted as Task : Duration : Toughness : Priority.
3	Dictionary Mapping Engine	Converts the raw task list into structured Python dictionaries mapping tasks to their [Duration, Toughness, Priority] values.
4	Optimization & Sorting Algorithm	Sorts tasks based on the user's toughness preference (Easy → Hard) and strictly enforces priority deadlines.
5	Output Display & Status Tracker	Returns task values to the user upon request and visually updates completed tasks (planned strikethrough implementation).

4 Algorithm / Flowchart / Program Design

Algorithm / Pseudocode

1. **Start**
2. Initialize empty task list `list_tasks = []`
3. Prompt user for subjective inputs (hosteller status, wake time, organizational schedule, break preferences).
4. **Loop** while True:
5. Prompt user for task input (Task:Duration:Toughness:Priority).
6. If input is 'end', **break** loop.
7. Split input string by ':' and append to `list_tasks`.
8. Remove trailing artifact (`list_tasks.pop()`) and format into Dictionary {Task : [Duration, Toughness, Priority]}.
9. Check conditional constraints (validate inputs, reject invalid values).
10. Sort dictionary based on combined Toughness and Priority weights.
11. Output the final schedule and task values.
12. **Stop**

Flowchart / Design



5 Python Concepts Used

Mention the Python concepts used in your project.

Python Concept	Where / How It Is Used
Variables and Data Types	Storing subjective inputs (strings), duration/toughness/priority (integers).
Conditional Statements	if store_tasks == 'end': break and restricting/validating user inputs.
Loops	while True: loop for continuous task data entry.
Lists / Tuples / Sets / Dictionaries	list_tasks.append() for raw storage; Dictionaries for mapping {Task : [D,T,P]}.
Functions	Modularizing the sorting engine and the input validation checks (To be implemented).
File Handling	
Exception Handling	
Libraries Used, if any	

6 Computational Feature

The primary computational feature of Schedailor is **scheduling and allocation**. The program does not merely store tasks; it mathematically evaluates a user's free time against task metadata. By taking the user's preference for "toughness progression" (e.g., doing hard tasks first) and weighting it against the actual "priority level / deadline" of the task, the algorithm computes an optimal order and allocates these tasks into the available free hours in the user's day, automatically resolving scheduling conflicts.

7 Important Outputs / Screenshots

Screenshot 1

```
Tasks : Duration to complete the task : Toughness level of the task(1-10) : Priority level  
of the task(1-10)Math:2:8:10  
Tasks : Duration to complete the task : Toughness level of the task(1-10) : Priority level  
of the task(1-10)end  
[['Math', '2', '8', '10']]
```

Screenshot 2

```
Tasks : Duration to complete the task : Toughness level of the task(1-10) : Priority level  
of the task(1-10)Math-2-8-10  
Tasks : Duration to complete the task : Toughness level of the task(1-10) : Priority level  
of the task(1-10)end  
[['Math-2-8-10']]
```

Screenshot 3

```
Tasks : Duration to complete the task : Toughness level of the task(1-10) : Priority level  
of the task(1-10)Math:2:8:11  
Tasks : Duration to complete the task : Toughness level of the task(1-10) : Priority level  
of the task(1-10)end  
[['Math', '2', '8', '11']]
```

8 Testing

Test	Input	Expected Output	Actual Output	Result
1	Math:2:8:10	Appends ['Math', '2', '8', '10']	['Math', '2', '8', '10']	Pass
2	end	Breaks loop, deletes last blank	Loop terminates	Pass
3	Math-2-8-10	Error: Invalid Request (Pr's Module)	['Math-2-8-10']	Fail
4	Priority 11	Error: Priority must be 1-10	Prompts re-entry	Fail
5	-	-	-	-

9 Team Member Contributions

Clearly mention the contribution of each team member.

Student Name	Enrollment No.	Contribution
Shreyan Gambhir	S26CSEU0814	Code Initiator
Priyanshu Kumar Bhardwaj	S26CSEU0841	Debugger
Bhupesh Singh	S26CSEU0727	Tester
Abhay Singh	S26CSEU0844	Analysar

10 Challenges Faced and Learning

- **Difficulties Faced:** Initially, managing collaborative development was a hurdle as team members encountered "Read-Only" and "Restricted Mode" file locks in VS Code after cloning the shared repository. Programmatically, a major challenge was preventing the while True loop from breaking when a user inputted an incorrectly formatted string (e.g., missing the colons).
- **How We Solved Them:** We resolved the VS Code roadblocks by explicitly trusting the workspace folder and standardizing our Git pull/push workflow. To fix the input crashes, we designed conditional validation checks (Pr's module) that intercept invalid requests and prompt the user to try again rather than crashing the script.
- **What We Learned:** We learned how to parse raw string inputs into indexable data using the .split(':') method, how to architect dictionary mappings ({Task : [D,T,P]}) for easier sorting, and how to effectively manage a multi-contributor codebase using GitHub.

11 Conclusion and Future Improvements

Schedailor successfully captures dynamic lifestyle variables and task metadata to generate highly tailored schedules. By shifting from standard 24-hour planners to a priority-and-toughness weighted system, it allows students to optimize their time based on psychological preferences and transit constraints. **Future Improvements:** Implementing file handling to save schedules permanently to TXT/CSV, and utilizing a strikethrough font logic on the terminal to live-update the status of completed tasks

12 References / Acknowledgements

Mention any websites, documentation, datasets, tutorials, libraries, or AI tools consulted during the project.

1. Python Documentation (String manipulation and Dictionary sorting methods).
2. Course materials on Computational Thinking (Loops, Conditionals, and Data Structures).

Declaration: We confirm that we understand the submitted project and have acknowledged the external resources used during development

